指令设计与功能验证

从指令规格解释八路乘法与两拍归约，展示 RTL 接入、正负数及边界测试，再检查代表性内部时序。

02 / 设计与实现 | 赛题 2.1、2.2、2.3.1 ~ 2.3.3
实验材料截至 2026-10-30 · 2026-11-02 第三版修订

目录

4 指令规格与微架构设计

4.1 固定配置与 custom-0 编码
4.2 按位掩码的八项求和与 128 位结果布局
4.3 接入 VFEX0 的两拍 VDot 单元
4.4 八路并行乘法与 8→4→2→1 归约树
4.5 两拍数据流：以 36 和 16 为例
4.6 VL=0 保持与精确状态更新
4.7 扩展到更高 SEW / VLEN

5 RTL 实现与整核接入

5.1 DecodeUnit：单微操作与 oldVd 依赖
5.2 FuConfig：把 VDot 挂入 VFEX0
5.3 VDot 模块：有符号乘法、+& 归约与 S1/S2 寄存器
5.4 末级：结果选择、非法检查与 redirect 取消
5.5 mtval 修复：ROB 保存编码字段，队首重建指令字
5.6 补丁、emu、参考库与测试镜像的对应关系

6 功能测试与补充时序验证

6.1 vdot-smoke：C 检查器逐字节判定 128 位目的
6.2 正数点积 case0 / case3：得到 36 与 16
6.3 负数点积 case2：−130048 只在低 32 位带符号
6.4 边界用例 case1/4/5/6：19 位和、零掩码、VL=0 与 VL=16
6.5 正常实例：输入到 FU 完成 2 周期，到退休 26 周期
6.6 取消实例：先完成的投机 VDot 未被提交
6.7 VS-dirty 请求与共享执行资源

附录 B 源码与验证索引

4 指令规格与微架构设计

vdot.vv 一次完成八路 signed INT8 乘积归约：低 32 位写精确和，高 96 位清零。标准 RVV 也能算到同一结果，但要拆成加宽乘、归约、配置切换和目的整理。我们把这条固定路径收进 VFEX0 里的专用两拍单元，寄存器依赖和精确异常仍走香山原有后端。

输入仍来自既有向量寄存器，结果仍由既有向量后端提交。这样我们可以单独研究算术融合带来的收益，同时保持处理器原有的依赖与精确异常机制。

本章对应赛题 2.1，说明软件可见行为、结构选择和数据流。各项设计的 Chisel 写法和后端注册方式集中放在第 5 章。

4.1 固定配置与 custom-0 编码

vdot.vv 使用 custom-0 机器码，在 SEW=8、LMUL=1、VLEN=128 的固定配置下工作，在本项目锁定版本的香山主译码表与 NEMU 顶层规则中，保存的静态检查未发现编码交叠。

设计基于香山昆明湖 V2 单核 MinimalConfig，VLEN=128，ROB 有 48 个槽位。赛题给定了三项要求：固定八项 signed INT8、归约初值为零、其余输出清零。在此基础上，我们进一步规定了结果写入哪些位、掩码怎样生效、VL 取哪些值合法，以及执行后状态怎样更新，形成下述实现规格。RTL 与参考模型共同遵守这份定义。

无掩码形式写为 `vdot.vv vd,vs2,vs1`，掩码形式追加 `v0.t`。当前工具链通过 `.word` 发出 custom-0 机器码。

表 4-1　指令编码

位段	字段	取值或含义
[31:26]	funct6	`000000`
[25]	vm	1：全部项参与；0：由旧 v0 的低八位使能
[24:20]	vs2	第二向量源
[19:15]	vs1	第一向量源
[14:12]	funct3	`000`
[11:7]	vd	向量目的
[6:0]	opcode	`0001011`，即 custom-0 `0x0b`

识别条件为 `(word & 0xfc00707f) == 0x0000000b`。例如 `vdot.vv v3,v2,v1` 编码为 `0x0220818b`，相应的掩码形式为 `0x0020818b`。保存的编码检查枚举了 65,536 种寄存器及 vm 组合，在锁定版本的香山主译码表与 NEMU 顶层规则中均未发现交叠〔1〕。

4.2 按位掩码的八项求和与 128 位结果布局

指令对源低八项做按位掩码的有符号乘积求和，结果符号扩展后写入低 32 位，高 96 位清零。

最低字节是源元素 0。对 i=0…7，把两个源的第 i 个字节按 8 位补码解释为 a_i、b_i；掩码使能为 m_i = vm OR old_v0[i]。指令计算：

```
S = Σ(i=0..7) mᵢ × aᵢ × bᵢ
vd[31:0]   = sign_extend_32(S)
vd[127:32] = 0
```

掩码按“位”选择参与归约的乘积。旧目的不参与求和。源寄存器高 64 位不参与运算；即使 VL=16，也只计算低八项。合法非零 VL 下，整个目的由本次结果定义：负数在低 32 位保留符号，高 96 位仍为零。

零掩码与 VL=0 是两种不同情况。零掩码是参与项为空的合法计算，结果全零；VL=0 则保留完整旧目的。这一区别要求硬件同时具备算术通路和 oldVd 保持通路。

表 4-2　配置和架构状态规定

主题	本实现规定
类型	SEW=8、LMUL=1、VLEN=128；有效向量上下文，vill=0，vtype 保留高位为零
长度	VL=0 保持目的；VL=8～16 固定计算八项；VL=1～7 非法
重启	vstart=0；非零 vstart 触发非法指令异常
掩码及重叠	支持 vm=0/1；允许源/目的重叠及掩码形式 vd=v0，读取旧版本输入
尾部与掩码策略	合法非零运算固定生成完整结果，vta/vma 不改变目的布局
成功状态	保持 vl、vtype、vxrm、vxsat；vstart=0；相关 VS 置 Dirty，包括合法 VL=0
非法状态	进入平台精确异常机制，目的向量及相应向量控制状态保持；陷阱 CSR 按异常路径更新

合法性检查排在 VL=0 保持分支之前，空长度同样要满足权限、类型和重启条件。指令要求 vstart 为零：它不保存中途的部分和，若从中间某个元素重新开始累加，得不到与完整执行相同的结果。完整规格及约定来源见指令规格基线和机器可读参数〔2〕〔3〕。

4.3 接入 VFEX0 的两拍 VDot 单元

新增算术硬件集中在 VFEX0 内的 VDot 单元；译码与执行配置还需注册该指令，ROB 和 CSR 增加异常指令字传递。重命名、发射、唤醒、写回和按序退休机制复用香山原有的向量后端。

一条 vdot.vv 译成一个微操作，经向量重命名和发射队列进入 VFEX0 内新增的 VDot 单元。两个向量源、oldVd、掩码和 VL/类型信息由既有的源读取与唤醒机制提供。输出沿原有向量结果通路写入物理寄存器，最终由 ROB 按序退休。



图 4-1　VDot 接入位置。新增部分是专用点积功能单元，寄存器版本管理、依赖检查和退休仍由既有后端承担。

这样，新增设计只需负责固定的点积运算和必要的控制。一个微操作完成全部计算，省去了标准指令组合中的多次配置和分步运算；相应地，必须向后端明确声明三件事：依赖旧目的、需等待掩码就绪、出错时上报的异常信息。这些接口由后续的功能测试和波形测试分别验证。

4.4 八路并行乘法与 8→4→2→1 归约树

4.4.1 树形归约与流水切分

八个乘积并行生成，经三级加法树归约；乘积之后和最终结果之后各设一级寄存器，接口固定为两拍。

固定八项输入允许一次并行生成全部乘积。归约采用 8→4→2→1 的树形结构，组合加法深度为三级。与依次串接七次相加相比，树形归约减少了单条组合路径经过的加法级数。代价是要并行布置八个乘法器及配套寄存器。

第一拍完成字节提取、乘法和掩码筛选，第二拍完成加法树与结果格式化。乘法与归约由此分在不同的寄存器区间。实际握手波形验证了这一时序，见第 6 章。我们没有把这项结构分析换算成综合频率或面积收益。

4.4.2 19 位精确和的位宽推导

最终和需要 19 位有符号数；每级保留进位即可在全范围内保持精确，无需饱和或舍入。

signed INT8 的范围为 −128…127。单项乘积最小为 −16256，最大为 16384，16 位有符号数足够表示。八项最大和为 `8×16384=131072`，而 18 位有符号最大值是 131071，所以最终归约必须保留第 19 位。

表 4-3　逐级扩位与保存位置

级	并行数量	位宽变化	数据含义
乘法及掩码	8	8×8 → 16 位	每项精确乘积，屏蔽项为零，进入 S1
配对相加	4	16+16 → 17 位	两项部分和
四项相加	2	17+17 → 18 位	四项部分和
八项相加	1	18+18 → 19 位	完整精确和
格式化	1	19 → 32 → 128 位	符号扩展至 32 位，高 96 位补零，进入 S2

负边界 −130048 的低 32 位为 `0xfffe0400`，正边界 131072 为 `0x00020000`。两者的高 96 位均为零。

4.5 两拍数据流：以 36 和 16 为例

有掩码和无掩码走同一条硬件，区别只在进入 S1 之前的项选择。

第一段：输入被接受后，从两个源取低八个字节，做有符号乘法。vm=1 时保留全部乘积；vm=0 时，未使能的项替换为 16 位有符号零。八个筛选后的乘积存入 S1，后续归约树只需对普通有符号数做加法。

第二段：从 S1 读出乘积，依次得到四个配对和、两个四项和与一个八项和，再符号扩展、补零并存入 S2。oldVd 经单独的保持路径同步传递，与有效位、向量控制信息和 ROB 身份保持同拍，最后在末级选择输出点积结果或 oldVd。

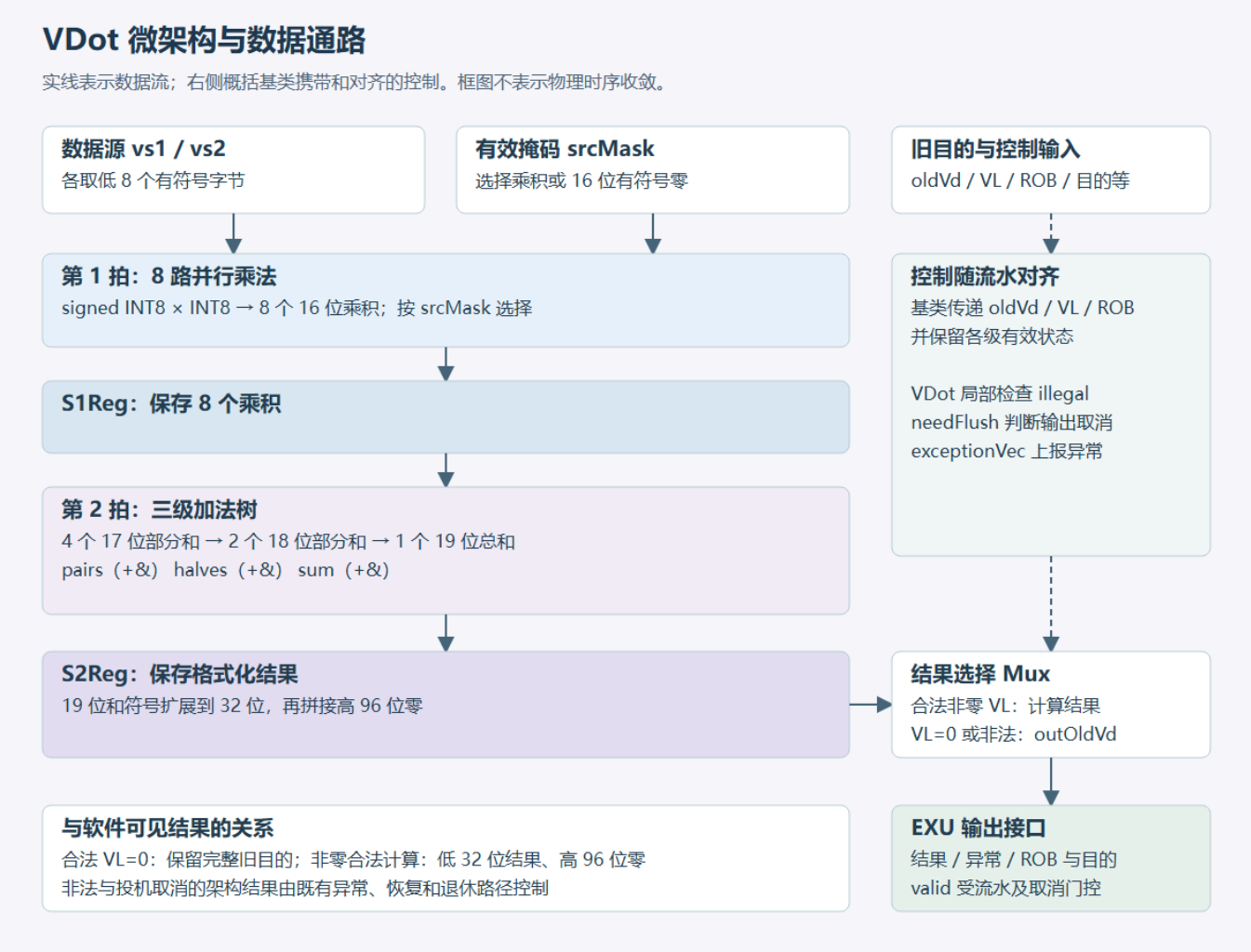


图 4-2　两拍点积数据通路。S1 保存筛选后的乘积，S2 保存格式化结果；oldVd 用于合法 VL=0 的保持。

以 A=[1,2,3,4,5,6,7,8]、B=[1,1,1,1,1,1,1,1] 为例：无掩码时得到 36；掩码 0x55 使第 0、2、4、6 项参与，得到 16。

表 4-4　同一输入的逐级计算

位置	vm=1	vm=0，mask=0x55
乘法后	[1,2,3,4,5,6,7,8]	[1,2,3,4,5,6,7,8]
S1	[1,2,3,4,5,6,7,8]	[1,0,3,0,5,0,7,0]
17 位配对和	[3,7,11,15]	[1,3,5,7]
18 位四项和	[10,26]	[4,12]
19 位总和	36	16
S2 低 32 位	0x00000024	0x00000010
S2 高 96 位	0	0

4.6 VL=0 保持与精确状态更新

4.6.1 VL=0 用 oldVd 把旧值复制到新物理寄存器

VL=0 仍要读取 oldVd：重命名之后，“保持目的”意味着把旧物理寄存器的值复制到新物理寄存器。

在乱序后端中，逻辑目的已被重命名到新的物理寄存器。若 v3 原来位于 P10，新指令获得 P51，那么 VL=0 的“保持 v3”就是把 P10 的值传给 P51，退休后 v3 才仍是原内容。单纯关闭结果写使能，新物理版本不会自动继承旧数据。

为此，译码声明 oldVd 依赖，执行端等待旧值就绪，在合法 VL=0 时选择该值输出。这里 oldVd 只用于完整目的保持，不参与非零点积的累加。合法 VL=0 完成后仍将 VS 置 Dirty，这是我们确定的状态策略。

4.6.2 结果只在按序退休后成为架构状态

结果可以先写入物理寄存器，但只有按序退休后才成为架构状态；被较老分支取消的结果不会提交。

ROB flag/value、物理目的和控制信息随数据一起前进。末级流水有效、且未被较老 redirect 取消时，单元才给出有效输出。非法配置通过异常位交给精确异常机制。若较老分支在结果已物理写入之后才纠正路径，后端恢复会阻止这一错误路径结果成为已提交的架构状态。

验证也要分三层：算术检查确认结果数值，FU 握手确认两拍执行，ROB/CSR 观察确认取消和状态更新。源码中的落实方式见第 5 章，实际输入和波形检验见第 6 章。

4.7 扩展到更高 SEW / VLEN

当前实现固定为 SEW=8、VLEN=128、8×INT8→INT32。提高 SEW 会增加乘法器和加法器位宽；归约树深度由项数 n 决定，固定 n=8 时仍为三级，掩码仍为每项一位。VFEX0 接入、oldVd 保持和精确异常的设计方式可继续采用，但数据宽度、时序和后端接口须重新核对。以下比较扩大元素位宽与按段复制两种扩展方向；它们是尚未实现、未验证的设计设想。

现在固定的部分　VDot 在展开时用 `require(VLEN == 128 && cfg.destDataBits == 128)` 和 `require(latency == 2 && latdiff == 0)` 锁定参数。合法配置只接受 SEW=8、LMUL=1。指令只读源低 64 位中的八个 INT8，精确和为 19 位，写入低 32 位，高 96 位清零。

随参数变化的部分　设每个结果归约 n 项、元素宽度为 SEW，各部件按表 4-5 变化。以 INT16→INT64 为例，八项 INT16 正好占满 128 位源，乘积为 32 位，八项最大和为 $8\times2^{30}=2^{33}$，需要 35 位有符号数。VL 的合法范围也要按 VLMAX=VLEN/SEW 重新规定。

表 4-5　参数化后各部件的变化（设计分析，无实测）

部件	一般形式	当前：SEW=8，n=8	示例：SEW=16，n=8
乘法器位宽	SEW×SEW → 2·SEW 位	8×8 → 16 位	16×16 → 32 位
归约树深度	平衡二叉树至多 $\lceil \log_2 n \rceil$ 级	3 级	3 级
精确和位宽	采用 $2\cdot SEW+\lceil \log_2 n \rceil$ 位可安全容纳	19 位	35 位
结果放置	结果容器宽度由扩展规格约定，须容纳精确和且不超过目的宽度	INT32 写 [31:0]，高 96 位清零	可选 INT64 写 [63:0]，高 64 位清零
源读取范围	n·SEW 位	低 64 位	完整 128 位
掩码粒度	每项一位，屏蔽项替换为 2·SEW 位零	old_v0 低 8 位，16 位零	old_v0 低 8 位，32 位零

可沿用的接口职责　扩展方案仍可按流水化 FU 组织，两源、oldVd、掩码、VL 输入以及配置和掩码唤醒继续承担原有职责。是否能保持一个微操作、一次目的更新和两拍唤醒，需要核对更宽的数据通路、寄存器堆分片、写回端口及实际时序；仅修改 `destDataBits` 并不能证明整核已支持更大 VLEN。合法 VL=0 仍应保持完整旧目的，非法条件仍交给精确异常通路。若编码字段不变，ROB 中的 mtval 重建方法可复用；若另设编码，则需同步修改识别与重建逻辑。

更大 VLEN 的方向：按 128 位段复制 8 路单元　以 SEW=8、VLEN=256 为例，可考虑两份八路单元：段 k 读取本段低八项，结果写入本段低 32 位，本段高 96 位清零；段 k 第 i 项使用 v0 的第 16k+i 位作掩码。这会产生两个段结果，属于新的多结果扩展语义，不能直接视为当前“整条指令只计算低八项、其余位清零”的 v0.1 规格在更大 VLEN 上的等价实现。实现前还须定义各段与 VL 的关系、未激活段的保持或清零、编码或版本区分，并同步修改 RTL、NEMU 和测试。本节只分析该候选方向：

- 每段仍为三级加法树，算术层级不因段数增加；更宽的总线、扇出和布线仍可能影响时序，两拍和 `CertainLatency(2)` 需重新验证。若改为跨全部元素归约，项数每翻倍才增加一级加法，流水切分须结合时序结果选择。
- 每段算术可沿用八项求和与结果格式化的做法；第 6 章的检查器和第 7 章的参考模型需增加多段输出、VL 和掩码测试，当前通过记录仍只覆盖 VLEN=128。
- 需要跨段总和时，软件可再用一次标准归约合并各段结果。这部分开销需另行测量，本文不作估计。

5 RTL 实现与整核接入

本章围绕五类关键改动展开：DecodeUnit 译码、FuConfig 配置、VDot 算术模块、ROB 中的 mtval 编码表和 CSR 选择器。完整接入还涉及指令模式、向量分类、FuType、执行拓扑和异常信息 Bundle；这些修改均包含在交付补丁中。本章按“译码 → 功能单元配置 → 算术和控制 → ROB/CSR”的顺序说明，对应赛题 2.2。

把点积公式写成算术模块只是接入的第一步。香山还需要知道指令属于哪种执行类型、读取哪些物理源、何时产生结果，以及异常时怎样保留故障身份。

源码以锁定的香山基版本叠加 VDot 及 mtval 修复补丁为准。下文为保存源码中的关键摘录，完整文件与补丁见阶段二复现索引。

5.1 DecodeUnit：单微操作与 oldVd 依赖

DecodeUnit 把 vdot.vv 译成一个微操作、一次目的更新，并声明 oldVd 依赖。

custom-0 位型被识别后，指令仍进入向量分类和权限检查通路。DecodeUnit 的设置如下：

```
when(inst.isVdot) {
  io.deq.isComplex := true.B
  io.deq.uopInfo.numOfUop := 1.U
  io.deq.uopInfo.numOfWB := 1.U
  io.deq.uopInfo.lmul := 1.U
}
```

`isComplex` 表示走相应的向量译码路径，`numOfUop=1` 让八项计算由同一个微操作完成。这里的 `uopInfo.lmul` 是内部微操作信息；向量类型中 LMUL=1 的编码则是 `vtype.vlmul=0`。

我们还在 `isDependOldVd` 条件中加入了 `inst.isVdot`，让调度端保留旧目的依赖。合法 VL=0 由此能把旧物理目的中的完整值复制到新物理目的，见第 4 章。oldVd 虽然不参与点积运算，仍需声明为源依赖，VL=0 时才能读到旧值。完整译码修改见 DecodeUnit 源码〔4〕。

5.2 FuConfig：把 VDot 挂入 VFEX0

`VdotCfg` 声明五组输入、固定两拍延迟和 128 位输出，VDot 由此挂入 VFEX0。

新增的 `FuType.vdot` 区分执行类型。执行拓扑把 VDot 挂入 VFEX0，`VdotCfg` 声明输入、延迟和输出能力：

```
  val VdotCfg = FuConfig (
    name = "vdot",
    fuType = FuType.vdot,
    fuGen = (p: Parameters, cfg: FuConfig) => Module(new VDot(cfg)(p)),
    srcData = Seq(Seq(VecData(), VecData(), VecData(), V0Data(), VlData())),
    piped = true,
    writeVecRf = true,
    writeV0Rf = true,
    latency = CertainLatency(2),
    vconfigWakeUp = true,
    maskWakeUp = true,
    destDataBits = 128,
    exceptionOut = Seq(illegalInstr),
  )
```

三组 `VecData()` 对应两个源和 oldVd，`V0Data()` 和 `VlData()` 提供掩码与长度信息。掩码与配置唤醒属性让指令等待正确版本的控制数据。`writeV0Rf` 支持 vd=v0，`destDataBits=128` 与完整目的布局一致。固定两拍延迟既规定了执行模块的流水级数，也让后端按两拍安排结果唤醒和写回。

这份配置决定算术模块的输入来源和结果写回方式。完整定义见 FuConfig 源码〔5〕。

5.3 VDot 模块：有符号乘法、+& 归约与 S1/S2 寄存器

VDot 模块的算术主体逐行对应第 4 章的数据通路：三级 +& 依次得到 17、18、19 位结果，S1、S2 两个寄存器划出两拍流水。

VDot 继承既有的 `VecPipedFuncUnit`，模块展开时检查 VLEN、结果宽度和延迟设置：

```
class VDot(cfg: FuConfig)(implicit p: Parameters) extends VecPipedFuncUnit(cfg) {
  require(VLEN == 128 && cfg.destDataBits == 128)
  require(latency == 2 && latdiff == 0)
```

算术主体如下：

```
  val products = VecInit((0 until 8).map { i =>
    val product = vs1(8*i+7, 8*i).asSInt * vs2(8*i+7, 8*i).asSInt
    Mux(srcMask(i), product, 0.S(16.W))
  })
  val p1 = S1Reg(products)
  val pairs = (0 until 4).map(i => p1(2*i) +& p1(2*i+1))
  val halves = (0 until 2).map(i => pairs(2*i) +& pairs(2*i+1))
  val sum = halves(0) +& halves(1) // 19 bits, including +131072
  val result = S2Reg(Cat(0.U(96.W), sum.pad(32).asUInt))
```

字节切片后的 `.asSInt` 保证 0x80 按 −128 参与乘法。掩码筛选发生在 S1 之前，屏蔽项为 16 位有符号零。每次 +& 都保留额外进位，三级结果依次为 17、18、19 位。`sum.pad(32)` 把有符号和扩展到 32 位，`Cat` 再拼接高 96 位零。负结果只在低 32 位带符号，不会污染高位。

`S1Reg`、`S2Reg` 让乘积与格式化结果分别跨过流水边界。输出的有效位、ROB 身份和控制信息由父类与末级逻辑同步传递，与结果数据处于同一拍。完整模块见 VDot.scala〔6〕。

5.4 末级：结果选择、非法检查与 redirect 取消

末级分别处理数据选择与异常标记：合法非零 VL 输出 S2 结果，合法 VL=0 或局部非法配置时数据选择 oldVd；非法配置另置 `illegalInstr` 并由精确异常路径阻止架构目的更新。被较老 redirect 覆盖的指令不产生有效输出。

末级检查执行时携带的向量配置，选择普通结果或旧目的，并把局部非法条件交给精确异常通路：

```
val illegal = outVecCtrl.vill || outVecCtrl.vsew =/= 0.U ||
  outVecCtrl.vlmul =/= 0.U || outVstart =/= 0.U ||
  (outVl =/= 0.U && (outVl < 8.U || outVl > 16.U))
io.out.bits.res.data := Mux(outVl === 0.U || illegal, outOldVd, result)
io.out.bits.ctrl.exceptionVec.get(illegalInstr) := illegal
// Keep vecWen/v0Wen for legal vl=0: the new physical destination needs oldVd.
// CSR updates and precise exceptions remain on the ordinary commit path.
io.out.valid := fixValidVec.last && !outCtrl.robIdx.needFlush(io.flush)
```

向量权限和部分类型检查位于上游，这里保留的是 FU 局部保护。

`io.out.valid` 同时要求末级有效、且未被较老 redirect 覆盖。ROB 指针中的 flag/value 区分环形队列的位置与代际。`needFlush` 据此判断本指令是否比发生跳转的分支更年轻、需要被取消。若结果已物理写入后才收到较老分支恢复，ROB 和重命名恢复会阻止它退休。因此，验证时分别观察物理寄存器写入和最终的架构状态更新。

向量脏状态（VS=Dirty）请求由同一拍各退休槽的标记汇总得到：

```
VS-dirty 请求来源 = isCommit AND OR(commitValid[i] AND dirtyVs[i])
```

无效退休槽即使残留 dirtyVs=1，也不参与聚合。各退休槽标记、ROB 注册请求与 CSR 接收请求之间的对应关系，用正常和取消两个实例检查，见第 6 章。

5.5 mtval 修复：ROB 保存编码字段，队首重建指令字

执行期非法异常现在也能把故障指令字写入 mtval：ROB 在分配时保存 VDot 的可变编码字段和代际，队首异常时重建指令字，再由 CSR 选择器优先接收。下面按现象、原因、改法、回归四步说明。

现象：DUT 的 mtval 为零　协同仿真中，短 VL 用例 short-1-vm0 由执行单元判为非法指令异常。参考模型的 mtval 为故障机器码 `0x0020818b`，DUT 的 mtval 为零。

原因：执行期非法异常拿不到指令字　原有 trapInstInfo 通路主要服务于译码期非法指令异常。VDot 的短 VL 非法条件在执行期才产生，指令到达 ROB 队首时找不回自己的编码。

改法：ROB 分配时保存可变编码字段和代际，队首重建指令字　新增表的核心源码如下：

```
val isVdot = RegInit(VecInit(Seq.fill(RobSize)(false.B)))
val generation = Reg(Vec(RobSize, Bool()))
val fields = Reg(Vec(RobSize, UInt(16.W)))
for (a <- io.alloc) {
  when(a.valid) {
    val idx = a.bits.robIdx.value
    isVdot(idx) := VdotInstructions.VDOT_VV === a.bits.instr
    generation(idx) := a.bits.robIdx.flag
    fields(idx) := Cat(a.bits.instr(25), a.bits.instr(24,20),
                       a.bits.instr(19,15), a.bits.instr(11,7))
  }
}
val f = fields(io.head.value)
val word = Cat(0.U(6.W), f(15), f(14,10), f(9,5), 0.U(3.W), f(4,0), "h0b".U(7.W))
val matched = isVdot(io.head.value) && generation(io.head.value) === io.head.flag
// Same one-cycle capture as Rob.io.exception; no writeback-order arbitration.
io.out.valid := RegNext(io.trap && io.illegal && !io.interrupt && matched, false.B)
io.out.bits := RegEnable(word, io.trap)
```

`vm`、`vs2`、`vs1`、`vd` 共 16 个可变位，加一个类型位和一个代际位，每槽 18 位；48 槽共 864 个存储字段位。固定的 funct6、funct3 和 opcode 可在异常时重建，无需逐槽保存完整的 32 位指令字。以上为字段容量估算，综合面积另需实测。

队首发生非法指令异常时，`matched` 确认类型和代际，`!interrupt` 排除中断，匹配信息随后与 ROB 异常信息同步寄存。CSR 用选择器优先接收该信息，其他异常保留原有通路：

```
csrMod.io.trapInst := xiangshan.backend.rob.VdotTrapInstSelect(
  csrIn.exception.valid, csrIn.exception.bits.isInterrupt,
  csrIn.exception.bits.exceptionVec(EX_II), csrIn.exception.bits.vdotTrapInst,
  trapInstMod.io.currentTrapInst)
```

整条路径为“分配时保存 → 队首异常匹配 → 重建指令字 → CSR 选择”。它按精确异常身份取值，不依赖各功能单元完成的先后顺序。源码入口为 VdotTrapInst.scala 和 CSR.scala〔7〕〔8〕。

回归：原异常用例 64 例 EXCEPTION 全部通过　修复版 emu 用原异常用例回归，64 例 EXCEPTION 全部通过，RTL 与参考模型的 mtval 一致。失败读数、故障 PC 和回归记录见第 7 章。

5.6 补丁、emu、参考库与测试镜像的对应关系

RTL 仿真器、NEMU 参考库和测试镜像分别编译，运行时配合使用。复现时需要对应的源码版本、补丁和二进制文件校验值。

译码、配置、VDot、ROB 和 CSR 修改共同进入 Chisel 展开和 emu 构建。NEMU 语义独立生成参考共享库，测试应用独立生成 RISC-V 镜像。

最终补丁已包含首版接入和异常指令字修复，见完整交付补丁〔9〕。后续功能、等价和性能实验复用锁定的修复版 emu 与源码 NEMU；只调整应用时，仿真器和参考库无需重新编译。下一章先用基础输入检查算术与布局，再用代表性波形检查它们在后端中的实际行为。

6 功能测试与补充时序验证

七个基础用例全部通过：正数、负数、正负极值、零掩码、VL=0 和 VL=16 均满足完整 128 位目的与状态要求。整核波形进一步表明，VDot 从 FU 输入到完成为两周期，结果按序退休；被较老分支取消的投机结果不会提交。

功能验证从正数、负数和边界用例展开，分别检查八项归约、补码运算，以及极值和特殊配置下的结果布局。各类用例都采用可手算的输入，并检查整个 128 位目的与相关状态。随后我们用整核波形观察通过功能检查的指令怎样执行、取消和退休。

本章前半部分对应赛题 2.3.1 ~ 2.3.3，后半部分为补充时序验证。异常矩阵、寄存器重叠及流水套件的整体差分结果在第 7 章汇总。

6.1 vdot-smoke：C 检查器逐字节判定 128 位目的

vdot-smoke 用 C 程序计算独立期望，逐字节比较全部 16 字节输出，并检查 VL、VS、vcsr 和 vstart；任一项失败即返回 1。

vdot-smoke 由 C 检查器与汇编包装组成。C 程序构造输入并计算独立期望；汇编装载源、掩码和旧目的，执行自定义机器码，捕获状态并存回完整结果。运行使用修复版单核 MinimalConfig、源码 NEMU 和固定 Flash，应用自检与 DiffTest 同时工作。

每个输入、旧目的和输出数组都按 16 字节对齐。旧目的填入 `0xa0…0xaf`，输出缓冲区填入 `0xee`；A、B 的高八字节分别填入 127、−128。这些非零初值用来检验旧值保持、输出是否真正写回，以及高半输入是否被误算。

汇编先以 e8/m1、VL=16 装载完整寄存器，再切到目标 VL。执行前把 VS 重设为 Initial，并令 vxrm=3、vxsat=1、vstart=0。目标指令执行后立即读取 VS 和 VL，之后才配置并存储完整 v3。这样，后续向量存储指令置起的 Dirty 不会被误记到目标指令名下。

C 检查器的核心判定如下，直接覆盖全部 16 字节：

```
for (unsigned i = 0; i < 16; ++i) {
  uint8_t expected = vl == 0 ? old[i] : i < 4 ? (uint32_t)sum >> (8*i) : 0;
  if (out[i] != expected) {
    printf("FAIL vdot case=%d byte=%d got=%x expected=%x\n",c,i,out[i],expected); return 1;
  }
}
```

非零 VL 下，低四字节保存小端 32 位点积，其余十二字节必须为零；VL=0 则逐字节与 old 比较。程序还要求 VL 与请求一致、VS=Dirty、vcsr=7、vstart=0。任一检查失败立即返回 1，全部用例通过才输出 PASS。实际执行顺序见完整 C 源码与汇编包装〔10〕〔11〕。

6.2 正数点积 case0 / case3：得到 36 与 16

两个正数用例均通过：无掩码得 36，掩码 `0x55` 得 16，高 96 位均为零。

无掩码用例 case0 取 A=`[1,2,3,4,5,6,7,8]`、B=`[1,1,1,1,1,1,1,1]`，VL=8、vm=1。八项乘积分别为 1…8，总和为 36，完整目的应为：

```
vd = 0x00000000_00000000_00000000_00000024
```

这个用例同时检查求和和高 96 位清零。旧目的预先为非零，只更新低位而漏掉高位也会失败。

case3 使用相同输入，改为 vm=0、mask=0x55。从最低位起只有 i=0、2、4、6 使能，期望变为 1+3+5+7=16，完整结果为 0x00000000_00000000_00000000_00000010。这个用例能区分按位掩码与错误的逐字节解释，并确认掩码作用于乘积本身。

表 6-1　正数测试的期望与记录

用例	输入与使能	期望点积	实测判定
case0	VL=8，vm=1，全部八项	36	sum=36 full128=OK VS=Dirty
case3	VL=8，vm=0，mask=0x55	16	sum=16 full128=OK VS=Dirty

6.3 负数点积 case2：−130048 只在低 32 位带符号

case2 通过，实测为 sum=-130048 full128=OK VS=Dirty：低 32 位为 0xfffe0400，高 96 位为零。

case2 的前八对输入均为 −128 与 127，单项乘积为 −16256，八项和为 −130048。若把 0x80 当作无符号 128，结果符号会出错；若归约位宽不足，也得不到精确结果。

19 位和符号扩展到低 32 位后为 0xfffe0400，再与高 96 位零拼接：

```
S  = 8 × (−128) × 127 = −130048
vd = 0x00000000_00000000_00000000_fffe0400
低地址起的字节 = 00 04 fe ff 00 00 00 00 00 00 00 00 00 00 00 00
```

逐字节比较同时验证了低 32 位的符号和高位清零；把负数符号扩展到整个 128 位的实现同样无法通过。该用例也覆盖了本运算的负极值。

6.4 边界用例 case1/4/5/6：19 位和、零掩码、VL=0 与 VL=16

表 6-2 中的边界用例全部通过；连同 6.2、6.3，七个用例的完整结果与指定状态检查均通过，整核运行同时给出 DiffTest、GOOD TRAP 和退出码 0。

6.4.1 case1：131072 检验第 19 位

case1 取八对 −128×−128，得到 8×16384=131072，低 32 位为 0x00020000。该值恰比 18 位有符号上界多 1，直接检验归约树是否保留了第 19 位。正、负极值共同覆盖了精确和的两端。

6.4.2 case4 / case5：零掩码清零，VL=0 保持

case4 为 VL=8、vm=0、mask=0，全部乘积被屏蔽，归约结果及整个目的应为零。

case5 为合法 VL=0，目的应保留 a0 a1 … af 全部旧字节，高位在左时为 0xafaeadac_abaaa9a8_a7a6a5a4_a3a2a1a0。它检验 oldVd 依赖及保持路径。日志仍会打印预先计算的参考字段 sum=36，但实际验收走的是代码中的旧值比较分支，这个字段不能当作 VL=0 的输出数值。

6.4.3 case6：VL=16 仍只算低八项

case6 把 VL 设为 16，低八项与 case0 相同，高八对特意填入 127×−128。按固定八项规格，结果仍为 36；若误算高半部，会额外加入 −130048。该输入能识别“按 VL 把全部十六项归约”的错误实现。

表 6-2　边界用例及其完整目的要求

用例	检查对象	完整目的要求	实际结果
case1	正极值、19 位和	低 32 位 `0x00020000`，高位零	通过
case2	负极值、符号扩展	低 32 位 `0xfffe0400`，高位零	通过
case4	零掩码	完整 128 位零	通过
case5	VL=0	完整旧目的保持	通过
case6	VL=16、高半部隔离	低 32 位 36，高位零	通过

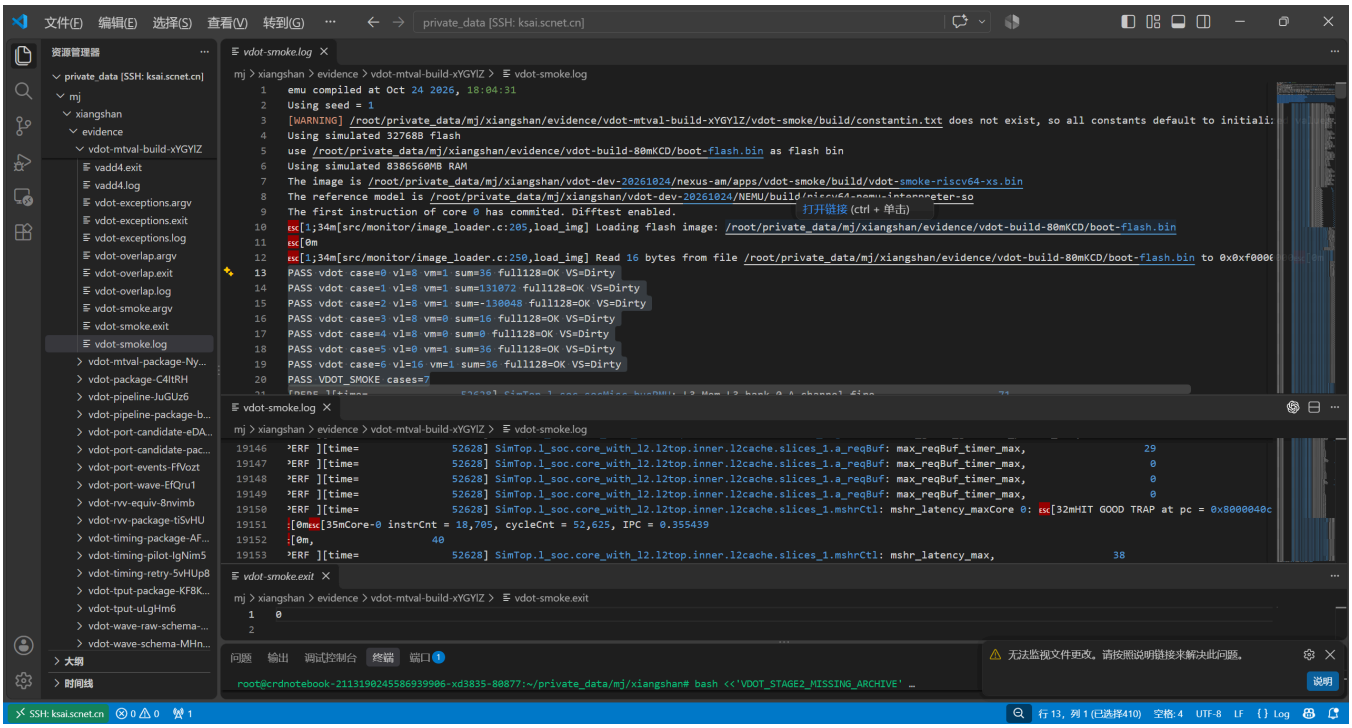


图 6-1　七例基础功能测试。正数、负数、极值、掩码及保持用例各自执行完整目的检查，原始记录见修复版整核回归〔12〕。

6.5 正常实例：输入到 FU 完成 2 周期，到退休 26 周期

正常实例从 FU 输入到完成为 2 周期，从输入到退休为 26 周期。前者是功能单元的流水延迟，后者还包含后端按序提交的等待。

基础结果通过后，我们用整核 VCD 检查内部时序。各窗口的时钟周期均为 2 ps，握手按事件沿前的稳定值判定。实例用 PC、机器码、完整 ROB flag/value、pdest 和分配区间关联；正文中的 ROB/pdest 均为十进制。

正常实例 PC=`0x80000208`，机器码=`0x0220818b`，ROB=1/36，pdest=62。它在 14208 ps 分配，14220 ps 接受 FU 输入，14224 ps 输出完整结果，14272 ps 在 ROB 退休。

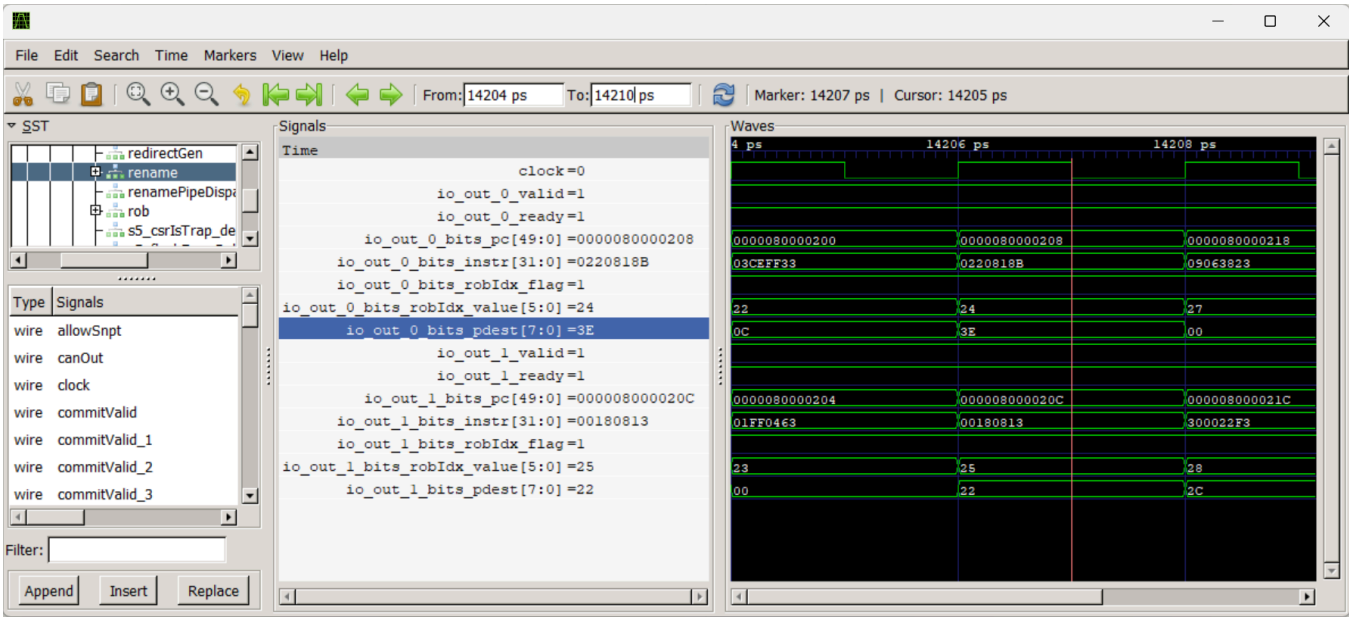


图 6-2　14207 ps 的分配沿前身份。波形中的 ROB 1/0x24、pdest 0x3e 对应正文的 1/36、62。

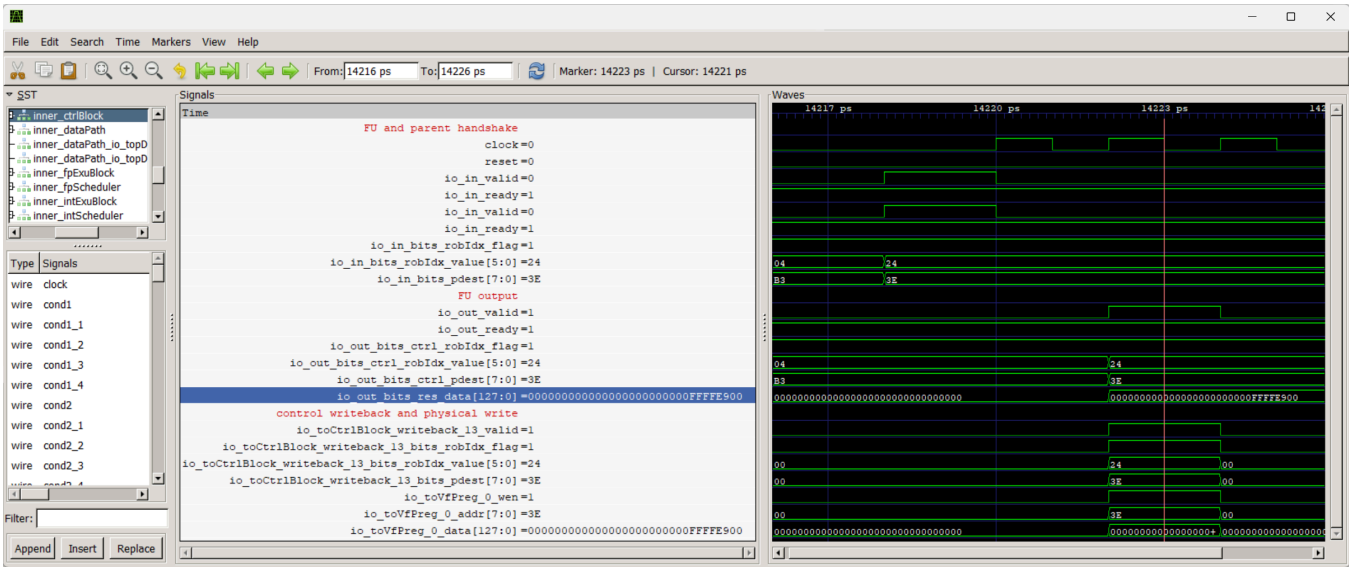


图 6-3　输入与输出相隔 4 ps，即 2 周期；输出为 `0x00000000000000000000000ffffe900`，低 32 位表示 −5888。

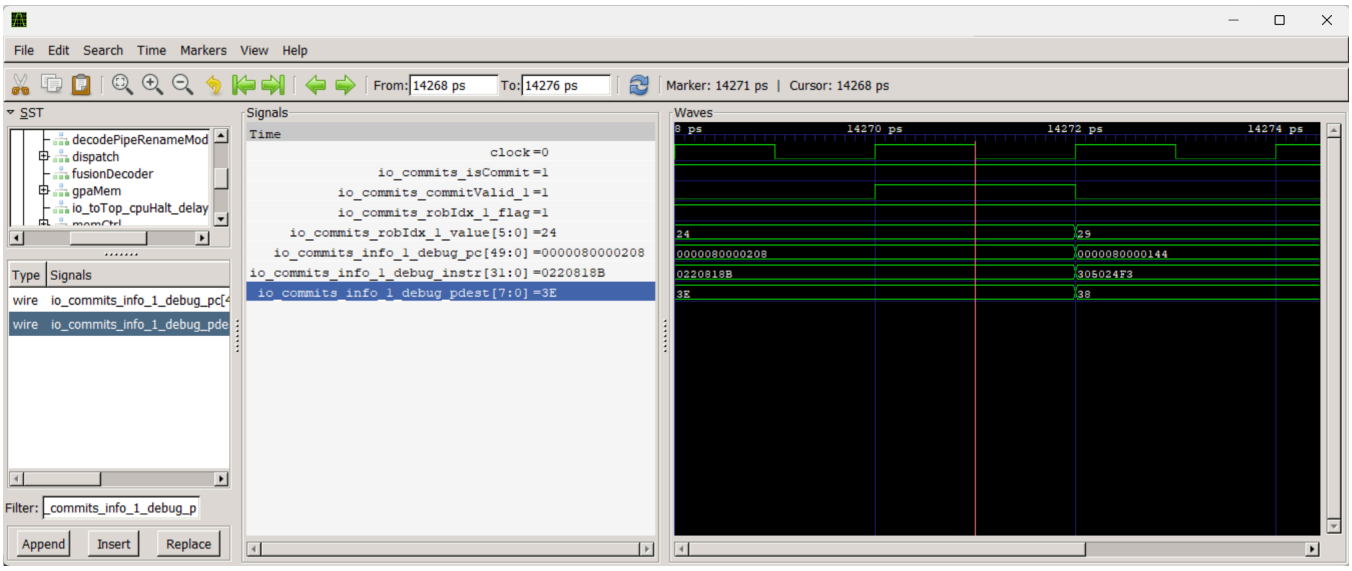


图 6-4　14271 ps 的 lane1 满足有效退休条件，ROB/pdest 与正常实例一致，对应 14272 ps 退休。

两个数字分别刻画功能单元流水和按序提交两段时间。原动态事件见正常与取消窗口核验〔13〕。

6.6 取消实例：先完成的投机 VDot 未被提交

投机 VDot 在 16908 ps 已完成计算；较老分支在 16952 ps 发出 redirect 后，直到原 ROB 身份于 16970 ps 被复用，这条 VDot 都没有有效退休。

取消实例与正常实例 PC 和机器码相同，但属于另一条动态指令：ROB=1/27，pdest=61。较老分支位于 PC=`0x80000204`，ROB=1/26。VDot 在较老分支结果确定之前执行，因而可能先产生物理结果。

表 6-3　投机实例的事件顺序

事件	时间	相对 VDot 输入周期
较老分支分配	16890 ps	−7
VDot 分配	16892 ps	−6
VDot 输入	16904 ps	0
VDot 完成	16908 ps	2
分支 redirect，覆盖年轻 VDot	16952 ps	24
分支退休	16958 ps	27
VDot 原 ROB 身份复用	16970 ps	33

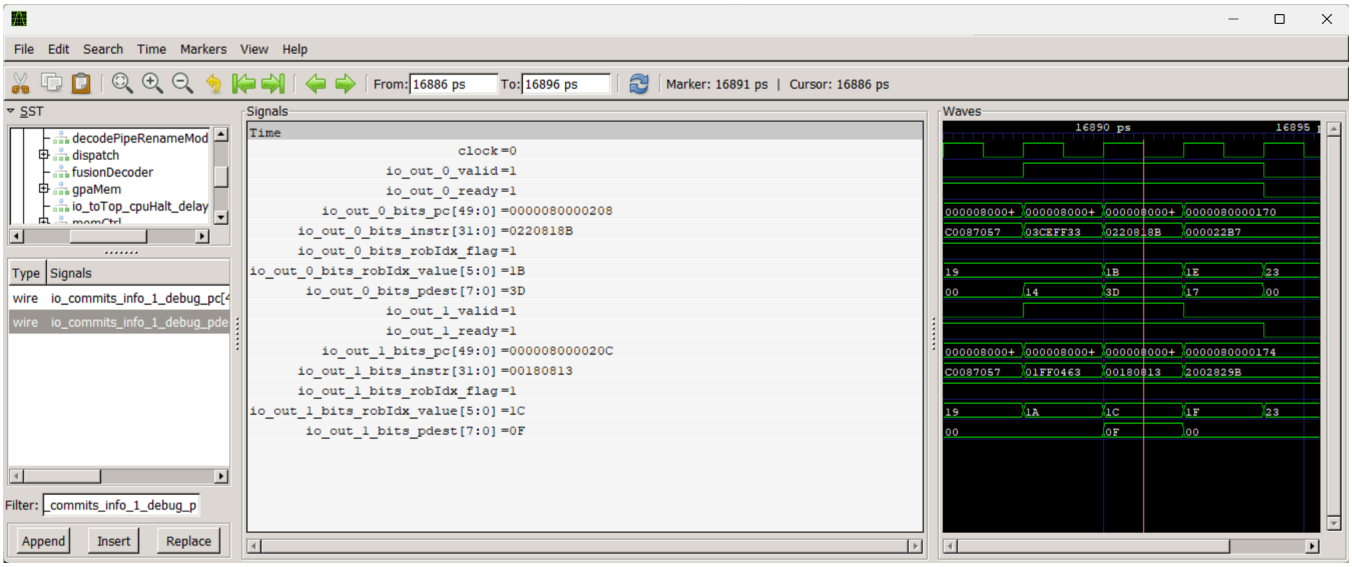


图 6-5　VDot ROB 1/0x1b、pdest 0x3d 在较老分支 ROB 1/0x1a 之后分配。

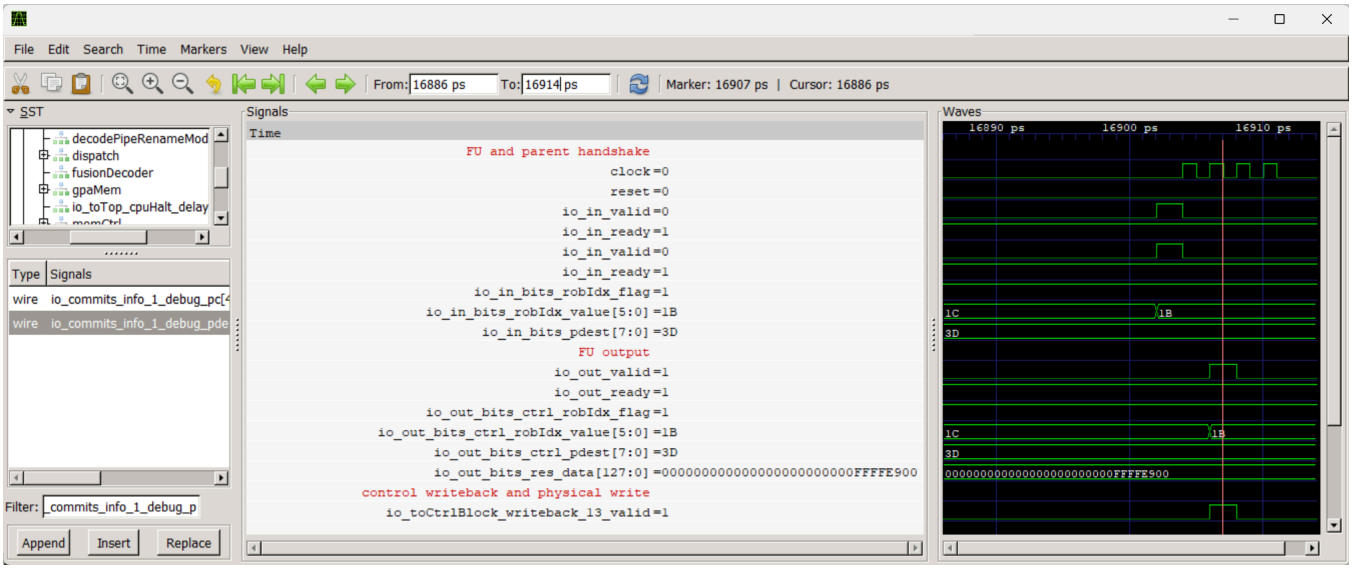


图 6-6　投机 VDot 在 16904、16908 ps 分别输入和完成，输出仍携带本次 ROB/pdest。

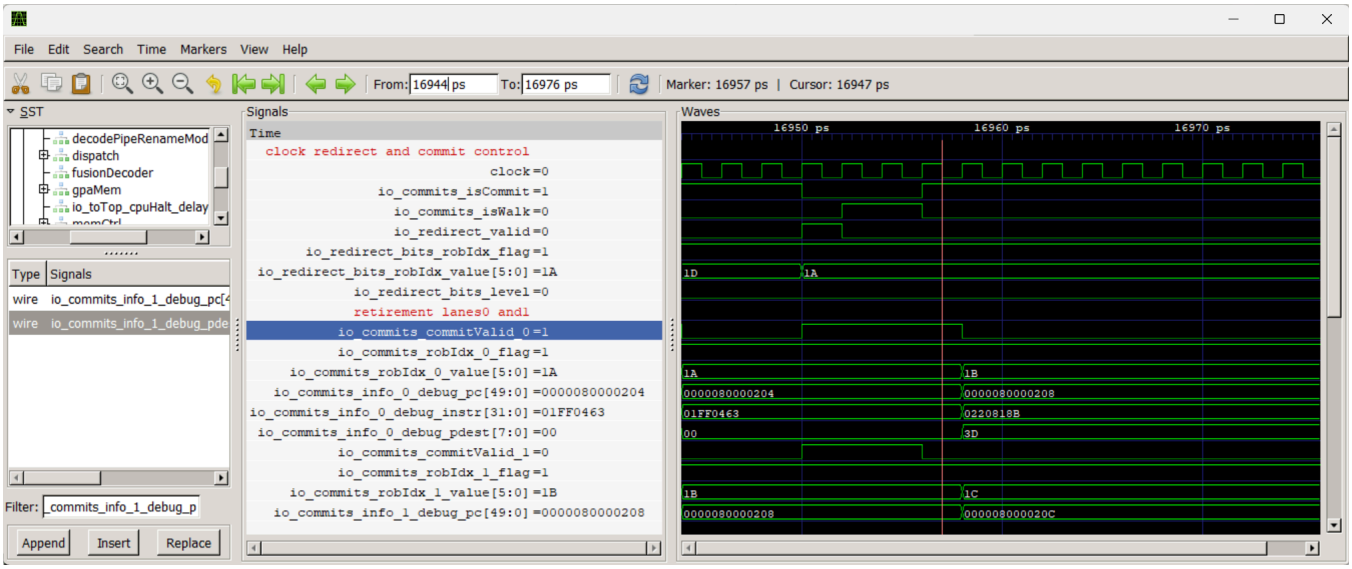


图 6-7　较老分支在 16952 ps 产生 redirect，随后退休；年轻 VDot 对应槽的身份数据没有形成有效退休。

FU 握手证明这条 VDot 已经算完；控制通知和物理端口观察补充了结果路径；ROB 则确认该投机结果没有提交。乱序执行允许先完成计算，架构状态仍由按序退休与分支恢复保护。

6.7 VS-dirty 请求与共享执行资源

VS-dirty 请求只来自有效退休贡献，2280 拍逐拍比较零差异。VDot 能与其他向量指令重叠推进；发射前端口竞争时，后者由仲裁延后两周期获准。

6.7.1 VS-dirty 只来自有效退休贡献

对保存窗口中的 2280 个非复位上升沿，我们逐拍比较 `isCommit AND OR(commitValid[i] AND dirtyVs[i])`、ROB 注册请求和 CSR 接收请求，结果零差异，其中八次正常 VDot 退休贡献均匹配。

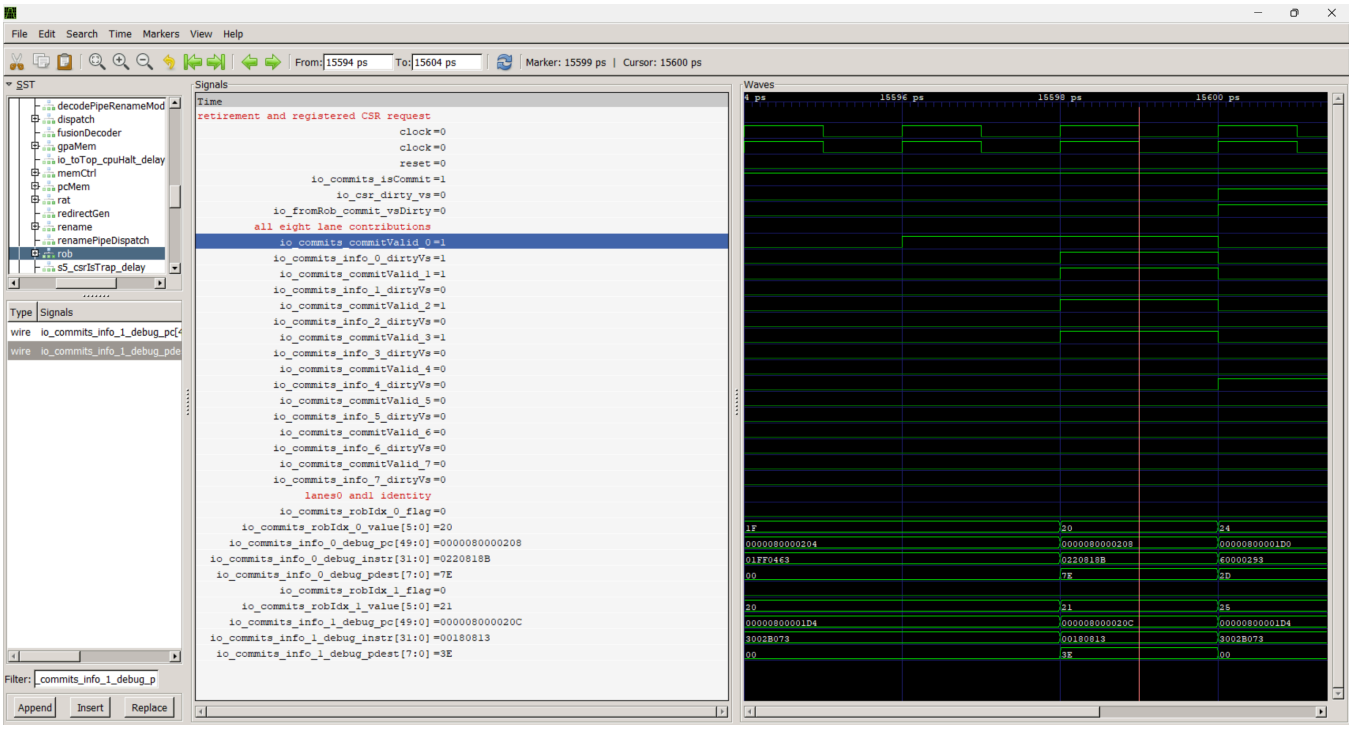


图 6-8　15599 ps 的 VDot 有效退休贡献在 15600 ps 沿后形成 ROB 与 CSR 请求。该例 ROB 为 0/32、pdest=126。

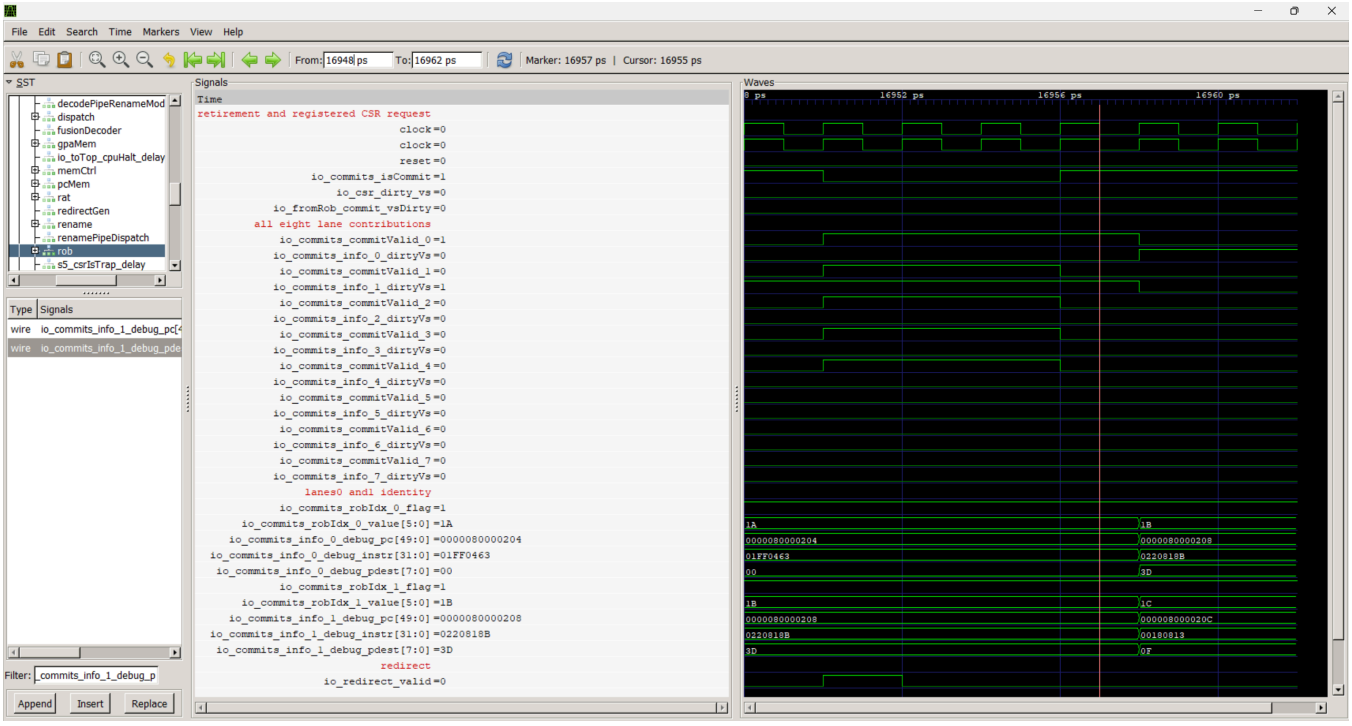


图 6-9　取消例中 VDot 槽虽然有 dirtyVs=1，但 commitValid=0，16958 ps 沿后请求保持为零。

两例说明，只有有效退休的槽才计入汇总，槽内残留的 dirtyVs 位不会单独触发请求。逐拍比较覆盖从退休槽标记到 CSR 接收请求的全过程；VS 寄存器有多个更新来源，它们同时出现时的优先级需另设用例检查。原记录见 CSR 请求核验〔14〕。

6.7.2 混发可重叠推进，端口竞争延后两周期获准

混发测试中，VADD 使用 EXU2，VMUL 和 VDot 使用 EXU0。三条指令的输入、完成和退休顺序如下：

表 6-4　混发实例，时间单位为 ps

指令	ROB / pdest	输入	完成	退休
VADD，PC=0x800001b8	1/35 / 62	13216	13218	13222
VMUL，PC=0x800001bc	1/36 / 63	13218	13222	13226
VDot，PC=0x800001c0	1/37 / 64	13220	13224	13228

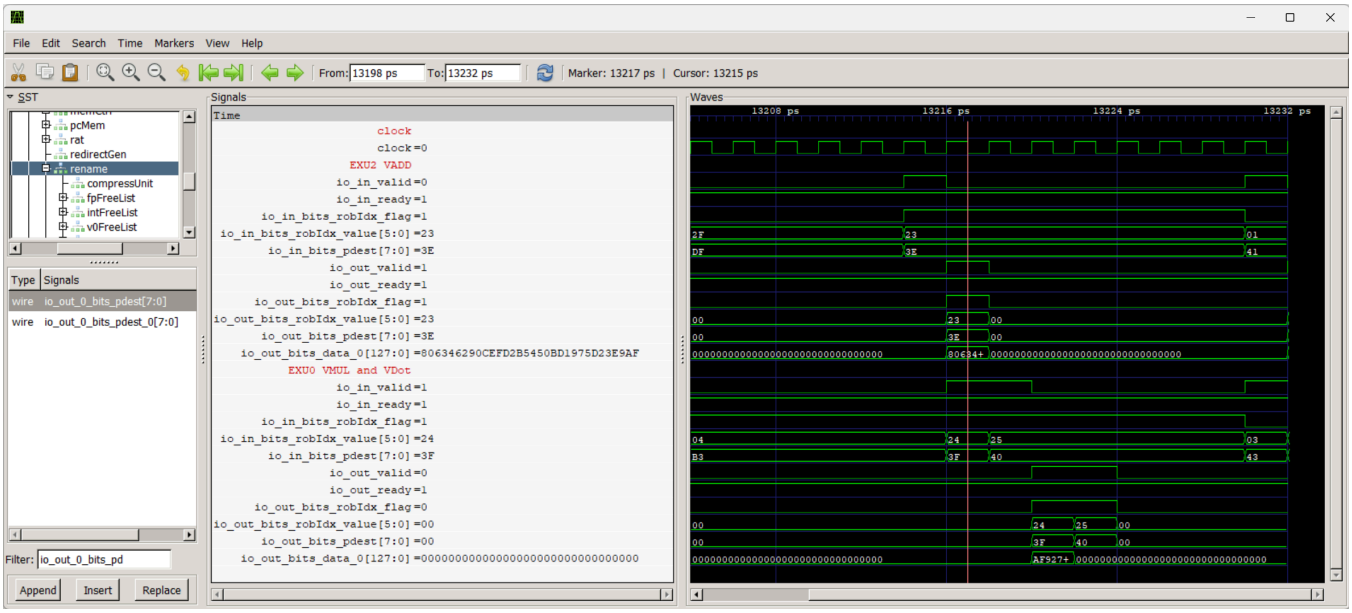


图 6-10　Marker=13217 ps 显示 VADD 输出与 VMUL 输入的有效条件，随后 VMUL、VDot 连续完成。

VMUL 与 VDot 的执行区间重叠，三者最终均退休，说明它们能在现有路径上交错推进。为进一步观察资源冲突，我们另构造了 VDot 与 VIOTA 共同申请发射前 VfWB 端口 0 的实例。

15658 ps 时两条请求同时有效：VDot（ROB=0/46，pdest=56）获准，VIOTA（ROB=0/47，pdest=57）的 ready 为零。VIOTA 于 15662 ps 再次申请并获准。两条指令随后分别在 15668、15672 ps 完成，在 15672、15676 ps 退休。

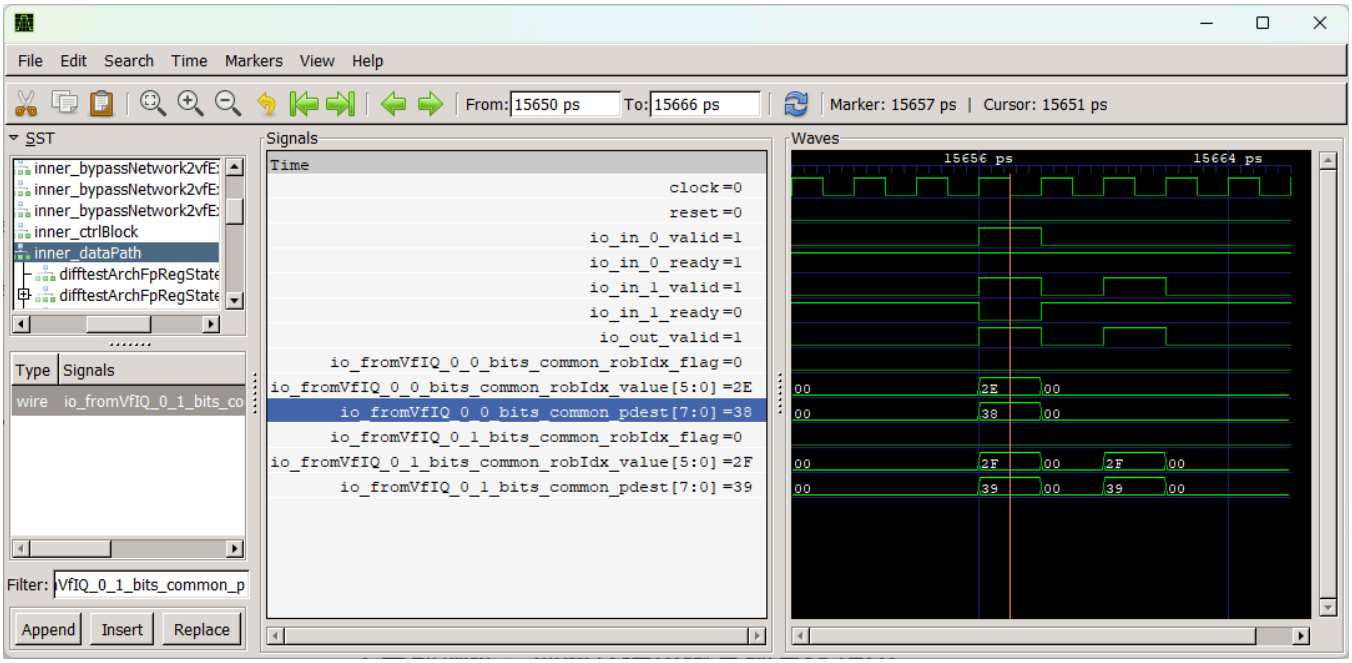


图 6-11　15658 ps 共同请求时，VDot 的 valid/ready 为 1/1，VIOTA 为 1/0，后者延后两周期获准。

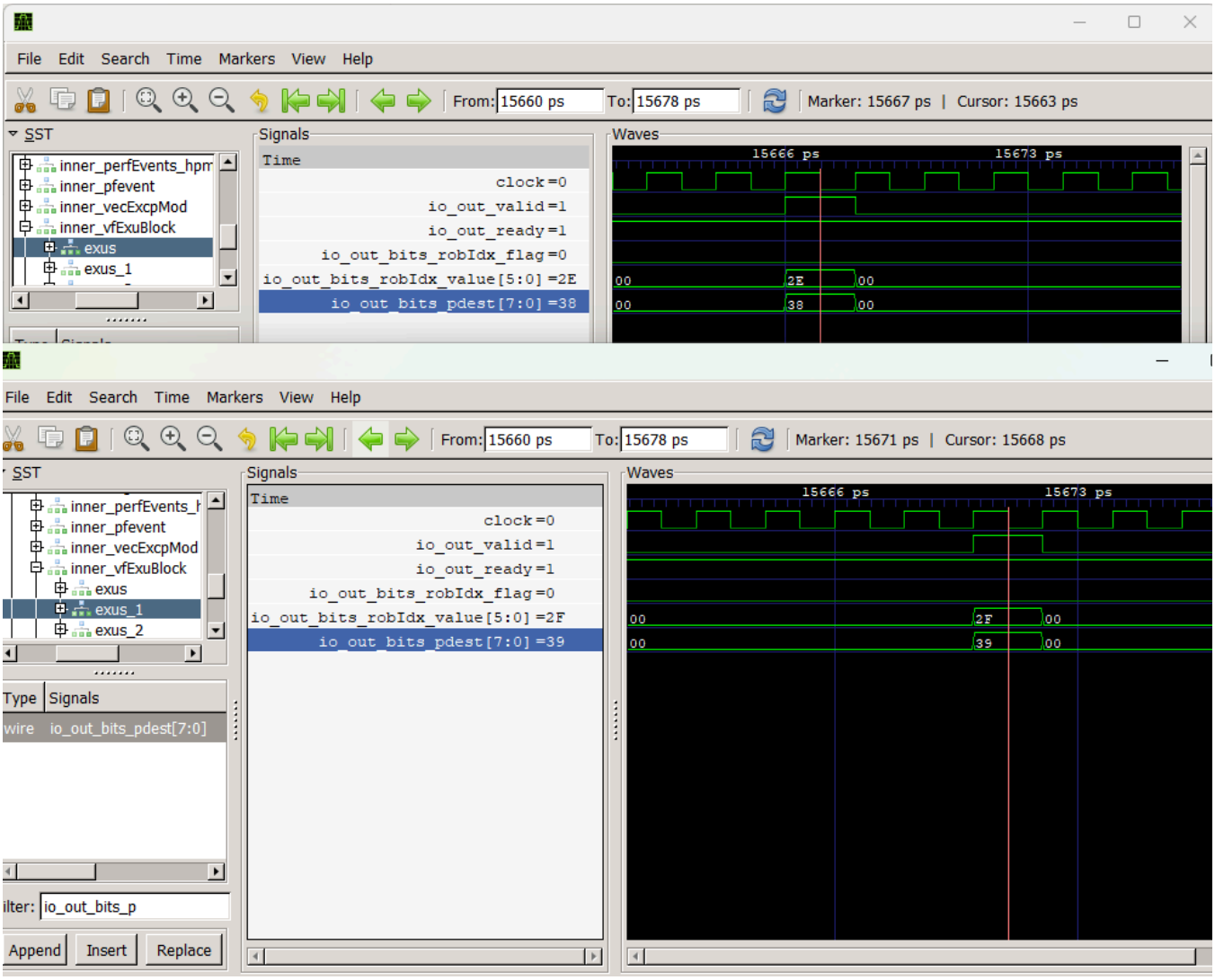


图 6-12　两条完成事件保持各自的 ROB/pdest，VDot 与 VIOTA 的结果在不同周期产生。

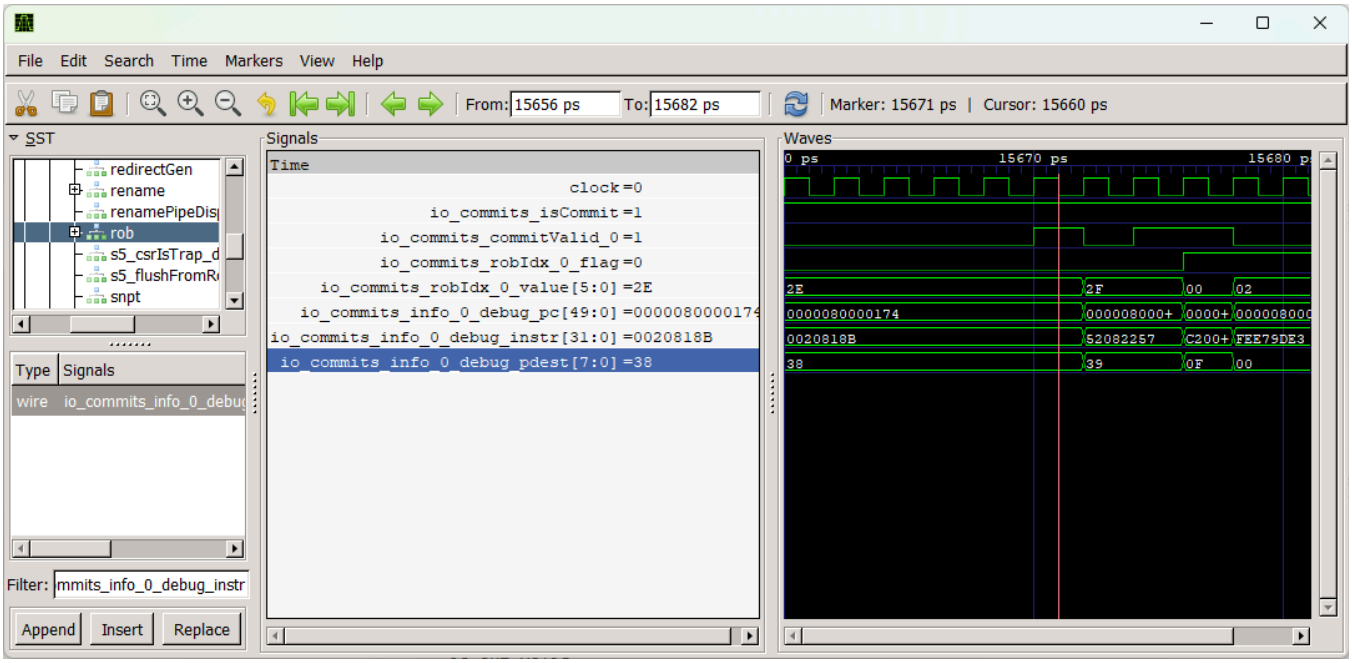


图 6-13　15671 ps 的 Marker 显示 VDot 退休沿前身份，后续时间窗显示 VIOTA 的退休身份。

两条指令的争用发生在发射前预约写回端口的阶段；该窗口中，真正写回结果时没有出现同一拍争用。本例验证了前端口仲裁及阻塞后的正常推进；写回同拍碰撞和更多 FU 组合留待后续用例。完整身份和采样记录见混发分析、端口事件与退休补图〔15〕〔16〕〔17〕。

基础测试确认了规定输入下的算术、布局与状态；补充波形把这些结果连接到真实的执行和提交事件。更大范围的异常、重叠与流水差分回归见第 7 章，那里还会用一个实际失配案例说明参考模型怎样帮助定位整核接入问题。

附录 B 源码与验证索引

本附录提供恢复方法与原件位置；正文只说明设计、关键源码和结果。第二阶段采用 SCNet Linux 服务器、单核 MinimalConfig、VLEN=128、ROB=48、EMU_TRACE=1。工具链记录包括 JDK 21、Mill 0.12.3、Verilator 5.052。

B.1 恢复源码和运行身份

XiangShan 基版本为 `16ae9ddcda54fc9a2fddffad73174cf793ac7814`。在独立 Linux 工程中恢复时，使用完整 RTL 补丁，并按 manifest 恢复指定依赖〔18〕〔19〕。完整补丁已包含首版接入与异常修复，不要再叠加首版补丁。

NEMU 基版本为 `a48a11f208ff81ad765591d83271d1b339fb3861`，使用 NEMU 补丁和保存的配置〔20〕。依次构建参考库、emu 和测试应用；在已有成功环境中重复测试时，只需按对应应用的需要构建。历史构建参数由构建脚本保存〔21〕。

服务器实验根为 `/root/private_data/mj/xiangshan`。修复版 emu 的相对路径为 `vdot-mtval-fix-20261024/XiangShan/build/emu`，源码参考库为 `vdot-dev-20261024/NEMU/build/riscv64-nemu-interpreter-so`，共同 Flash 为 `evidence/vdot-build-80mKCD/boot-flash.bin`。

表 B-1　成功运行保存的关键 SHA256

文件	SHA256
emu	`f9653532fd2a4b9f0d0cc08c131f7c258d4b62c46e672c6886d6643aad926850`
NEMU 参考库	`11b9f6807c36ed860e0c64cc4e58d27216b19335ecde1ce5797ba2d1cd057b74`
NEMU 配置	`64282bca051b5dbc5fca742f4fbca2ade937a70f3c4d2ec93c1f4836aca5d9be`
Flash	`fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724`

这些是归档的运行身份；本次重写没有重新构建，也没有对服务器当前文件重新取哈希。原身份记录见运行文件校验清单〔22〕。

B.2 功能与动态事件材料

内容	工程入口
指令规格与参数	规格〔2〕、JSON 参数〔3〕
RTL 修改文件	源码快照〔23〕
正负数及边界程序	测试源文件目录〔24〕
基础、异常及重叠回归	修复回归记录〔12〕
正常与取消实例	原事件〔25〕、完整窗口核验〔13〕
退休到 CSR 请求	逐拍核验〔14〕
混发与路由	混发事件〔15〕、路由核对〔26〕
发射前端口竞争	端口事件〔16〕、退休身份补图〔17〕
完整事件及 VCD 归档	归档目录〔27〕

基础功能测试与代表性波形分别对应第 6 章列明的输入和窗口。未覆盖的取消时刻、实际结果写回同拍碰撞或其他资源组合，需要对应的独立用例。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	编码检查	docs/reviews/vdot-encoding-audit-2026-10-19.md
2	指令规格基线	docs/vdot-spec-v0.1.md
3	机器可读参数	specs/vdot-v0.1.json
4	DecodeUnit 源码	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/decode/DecodeUnit.scala
5	FuConfig 源码	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/FuConfig.scala
6	VDot.scala	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/wrapper/VDot.scala
7	VdotTrapInst.scala	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/rob/VdotTrapInst.scala
8	CSR.scala	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2/src/main/scala/xiangshan/backend/fu/wrapper/CSR.scala
9	完整交付补丁	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2
10	C 源码	artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke/main.c
11	汇编包装	artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke/vdot.S
12	修复版整核回归	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/review.json
13	正常与取消窗口核验	artifacts/vdot-window-supplement-2026-10-25/server-fgpWCe/review.json
14	CSR 请求核验	artifacts/vdot-csr-commit-window-2026-10-25/server-aReAEz/review.json
15	混发分析	artifacts/vdot-mixed-resources-2026-10-25/server-9k3waS/review.json
16	端口事件	artifacts/vdot-port-events-2026-10-25/server-FfVozt/review.json
17	退休补图	artifacts/vdot-gtk-review-2026-10-26/step3-retirement-instruction/review.json
18	完整 RTL 补丁	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/XiangShan-full.patch
19	manifest	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/manifest.json
20	NEMU 补丁	artifacts/vdot-integration-2026-10-24/delivery/payload/NEMU.patch
21	构建脚本	artifacts/vdot-mtval-fix-2026-10-24/delivery-v2/build_scnet_vdot_mtval.sh
22	运行文件校验清单	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/accepted-identities.sha256
23	源码快照	artifacts/vdot-mtval-fix-2026-10-24/patch-fixture-v2
24	测试源文件目录	artifacts/vdot-integration-2026-10-24/delivery/payload/vdot-smoke
25	原事件	artifacts/vdot-window-events-2026-10-25/server-UbYWTP/review.json
26	路由核对	artifacts/vdot-mixed-route-audit-2026-10-25/server-ApJqcC/review.json
27	归档目录	artifacts/stage12-archive-audit-2026-10-27/server-LpAu2Y